Tutorial:

Calling functions

Functions calls take the form:

```
add sum $1 $2
```

This would mean `sum := 1 + 2`.

The `$` sigil is used for variable names.

Enums

Enums types represent a choice between multiple discreet values. They are defined such as:

```
enum Bool true false
```

The variants can then be referenced by their names using the `#` sigil:

```
id my_true #true
id my_false #false
```

Variants can be matched against with `#`:

```
# result $my_bool true;$1 false;$0
```

Notice that the hash sigil is not used for the left side of the semicolon.

Unions

Unions are similar to enums except that each variant comes with exactly one attached value.

Unions are discriminated. They are defined such as:

```
union NatOption nat_some:Nat nat_none:Unit
```

The variants can then be referenced by their names plus a value of the correct type using the percent sigil:

```
id none %nat_none#unit
id my_some_5 %nat_some$5
```

Variants can be matched against with `%`, with the right side of the semicolon being unary functions that are applied rather than just values.

Structs

Structs combine types by having multiple fields. They are defined such as:

```
struct NatPair fst:Nat snd:Nat
```

They can then be constructed on their own line such as:

```
NatPair my_pair fst;$5 snd;$3
```

Members can then be accessed with a dot:

```
id my_fst $my_pair.fst
```

Defining Functions

Functions can be defined like this:

```
fn increment Nat n:Nat
    add ~ $n $1
```

The header is fn, the name of the function, the return type, and then colons of the parameter names and their types. The body is indented four spaces. The tilde in the return variable position of add means to return the value from the function. This function can then be called like any other:

```
increment res $3
```

Note that `increment` is also a variable and can be referenced as `$increment`:

This variable has the type `<Nat,Nat>`, where the first type name represents the return type, and the rest represent the parameter types.

Program 1: Hello, world!

```
list_13 str $'H $'e $'l $'l $'o $'comm $'- $'w $'o $'r $'l $'d $'excl
println _ $str
```

A return name of underscore tells the language to not store the result. Here the underscore is used because println returns Unit, and we don't need to store it.

The ? Type

The language does not have type variables or polymorphism. Instead, the ? "type" tells the language to allow any type. They can be used to define types for struct and unit:

```
struct Pair fst:? snd:?
Pair bool_pair fst;#true snd;#false
Pair nat_pair fst;$5 snd;$3
```

They can also be used in functions:

```
fn if ? cond:Bool if_true:? if_false:?
    # ~ $cond true;$if_true false;$if_false
```

Here the expectation that `if_true` and `if_false` have the same type is not checked by the function: it is checked by the enum match.

Program 2: Fizzbuzz

```
id times $32

list_4 fizz_str $'F $'i $'z $'z
list_4 buzz_str $'B $'u $'z $'z
concat fizzbuzz_str $fizz_str $buzz_str
fn loop Unit n:Nat
    show_nat n_str $n
    divides is_fizz $n $3
    divides is_buzz $n $5
    if when_not_fizz $is_buzz $buzz_str $n_str
    if when_fizz $is_buzz $fizzbuzz_str $fizz_str
    if str $is_fizz $when_fizz $when_not_fizz
    println ~ $str
range nums $1 $times
for_each _ $loop $nums
```

Laziness

Functions with no parameters can be used for lazy computation. When used in this way, they are called thunks. These are especially important because usual control flow structures are missing. The most common use of laziness is to avoid unnecessary calls to expensive functions. For example, in the program Fizzbuzz, `show_nat` is called 55 times, but in Fizzbuzz Lazy, it is only called 31 times, a 44% decrease.

Program 3: Fizzbuzz Lazy

```
id times $32

list_4 fizz_str $'F $'i $'z $'z
list_4 buzz_str $'B $'u $'z $'z
concat fizzbuzz_str $fizz_str $buzz_str
just get_fizz_str $fizz_str
just get_buzz_str $buzz_str
just get_fizzbuzz_str $fizzbuzz_str
fn loop Unit n:Nat
    delay1 get_n_str $show_nat $n
    divides is_fizz $n $3
    divides is_buzz $n $5
    if get_when_not_fizz $is_buzz $get_buzz_str $get_n_str
    if get_when_fizz $is_buzz $get_fizzbuzz_str $get_fizz_str
    lazy_if str $is_fizz $get_when_fizz $get_when_not_fizz
    println ~ $str
range nums $1 $times
for_each _ $loop $nums
```

The thunks in this program are prefixed with `get_`.